Sampling Techniques & Measures of Central Tendency

Session Objectives:

- Understand data collection methods & sampling techniques.
- Understand and apply different sampling techniques (Random, Stratified, Cluster, Systematic).
- Identify and explain sampling bias & errors and their impact.
- Calculate & interpret mean, median, and mode.
- Use Python to demonstrate sampling and compare sample means with population mean.

Color	Satisfaction	Temperature	Income	Children	Weight
LightSlateGray	Medium	81°F	$110.25	5	7.46 kg
MediumSlateBlue	High	29°F	$968.69	7	1.86 kg
Indigo	Medium	70°F	$451.59	9	9.56 kg
BurlyWood	Medium	28°F	$815.61	7	1.97 kg
SeaGreen	Medium	74°F	$681.58	3	5.26 kg
Nominal	Ordinal	Interval [No True Zero]	Ratio [True Zero]	Discrete	Continuous

```python
# Categorical Column with No Order -> Nominal
# Favourite Color

import numpy as np
import pandas as pd

np.random.seed(42) # reproducibility
fav_col = ['orange','red','purple','blue','yellow','green','pink','violet','black','white','cyan']
nominal_data = np.random.choice(fav_col , 1000)
df_nominal = pd.DataFrame(nominal_data , columns = ['Fav Color'])
df_nominal
```

	Fav Color
0	pink
1	blue
2	cyan
3	violet
4	yellow
...	...
995	white
996	black
997	red
998	violet
999	violet

1000 rows × 1 columns

```python
df_nominal['Fav Color'].value_counts()
```

```
Fav Color
orange    110
purple    103
yellow    102
white     100
violet     90
black      88
green      87
pink       86
blue       84
red        79
cyan       71
Name: count, dtype: int64
```

```python
# Categorical Column with Order -> Ordinal
# Satisfaction_Level

np.random.seed(11) # reproducibility
satisfaction_level = ['Very Poor 😖', 'Poor 😫', 'Average 😅' , 'Good 😁' , 'Awesome 😍']
ordinal_data = np.random.choice(satisfaction_level , 1000)
df_ordinal = pd.DataFrame(ordinal_data , columns = ['Satisfaction_level'])
df_ordinal
```

	Satisfaction_level
0	Poor 😫
1	Very Poor 😖
2	Good 😁
3	Poor 😫
4	Awesome 😍
...	...
995	Average 😅
996	Good 😁
997	Good 😁
998	Very Poor 😖
999	Poor 😫

1000 rows × 1 columns

```python
df_ordinal['Satisfaction_level'].value_counts()
```

```
Satisfaction_level
Good 😁          219
Average 😅       204
Very Poor 😖     198
Poor 😫          191
Awesome 😍       188
Name: count, dtype: int64
```

```python
# Quantative Date [Numerical] -> Fahrenheit [Interval]
# Histogram
import matplotlib.pyplot as plt

interval_data = np.random.randint(0,100,1000)
df_interval = pd.DataFrame(interval_data , columns = ['Fahrenheit'])
df_interval
```

	Fahrenheit
0	83
1	56
2	99
3	57
4	91
...	...
995	19
996	91
997	56
998	51
999	26

1000 rows × 1 columns

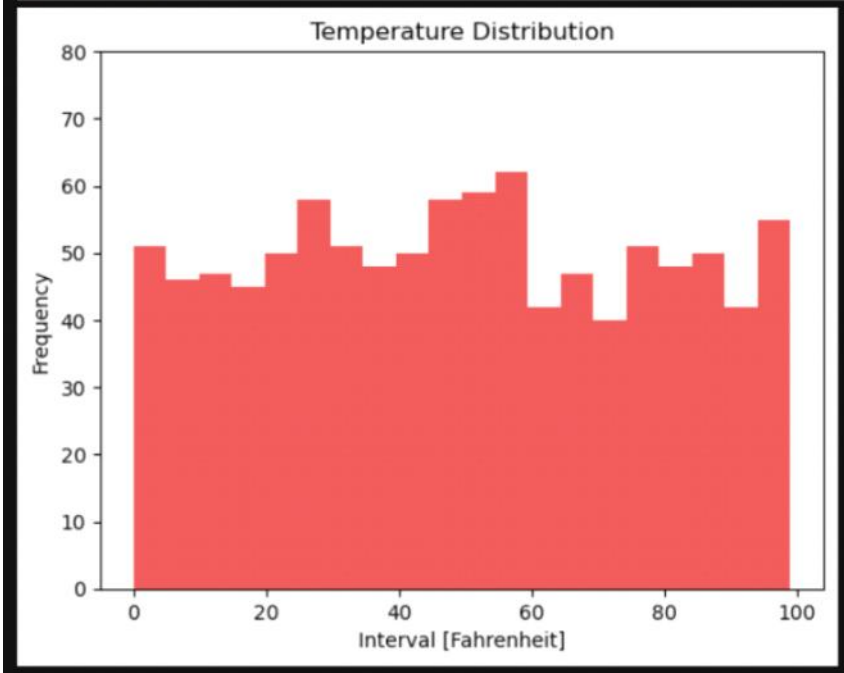
```python
df_interval['Fahrenheit'].value_counts()
```

```
Fahrenheit
26    21
46    18
27    17
35    16
99    15
      ..
5      5
38     4
1      4
65     4
29     3
Name: count, Length: 100, dtype: int64
```

```python
df_interval.describe()
```

	Fahrenheit
count	1000.000000
mean	49.287000
std	28.500332
min	0.000000
25%	26.000000
50%	49.000000
75%	74.000000
max	99.000000

```python
# Histogram
plt.hist(df_interval['Fahrenheit'] , bins = 20, color = '#ED5A5A')
plt.title("Temperature Distribution")
plt.xlabel("Interval [Fahrenheit]")
plt.ylabel("Frequency")
plt.ylim(0,80)
plt.show()
```




```python
# Quantative Date [Numerical] -> Income [Ratio]
ratio_data = np.random.rand(1000) * 1000
ratio_data = ratio_data.round(0)
ratio_data
```

```
array([766.,  68., 648., 199., 492., 810., 831., 739., 645., 719., 110.,
       297., 132.,  63., 164., 157.,  59., 966., 364., 378., 931., 794.,
       691., 709.,  57., 248., 165., 235., 153., 128., 328., 676., 728.,
       488., 337., 980., 930., 143., 558., 489., 840., 384., 198., 836.,
       137., 677., 719., 500.,  87., 755., 547., 183., 359.,  51., 286.,
       106., 651., 667., 111., 235., 580., 464., 255., 987., 701., 215.,
       813., 537., 573., 552., 582., 655., 639., 501., 564., 729.,   4.,
       403.,  29., 615., 709., 382., 126., 478., 476., 626., 148.,  46.,
       527., 322., 794., 595., 699., 310., 858., 991., 677., 235., 244.,
       830., 247., 916., 366., 505., 116., 315., 283., 481., 927., 819.,
       311., 833., 464., 699., 266., 167., 198., 933., 636., 403.,  79.,
       589., 206., 539., 677., 421., 267., 158., 183.,  70., 986., 874.,
       880., 856., 604., 524.,  84., 604., 957., 365., 663., 570., 622.,
       710., 820., 140., 231., 483., 460., 925., 287.,  50., 974., 910.,
       159., 381., 930., 557., 427., 156., 347., 307., 860., 216., 670.,
       130., 460., 658., 738., 764., 886., 169., 851., 194., 476., 775.,
       990., 746., 461., 943., 653., 940., 748., 360., 344., 796.,  65.,
```

```python
type(ratio_data)
```

```
numpy.ndarray
```

```python
ratio_data.shape
```

```
(1000,)
```

```python
df_ratio = pd.DataFrame(ratio_data , columns = ['Income'])
df_ratio
```

	Income
0	766.0
1	68.0
2	648.0
3	199.0
4	492.0
...	...
995	81.0
996	152.0
997	876.0
998	540.0
999	154.0

1000 rows × 1 columns

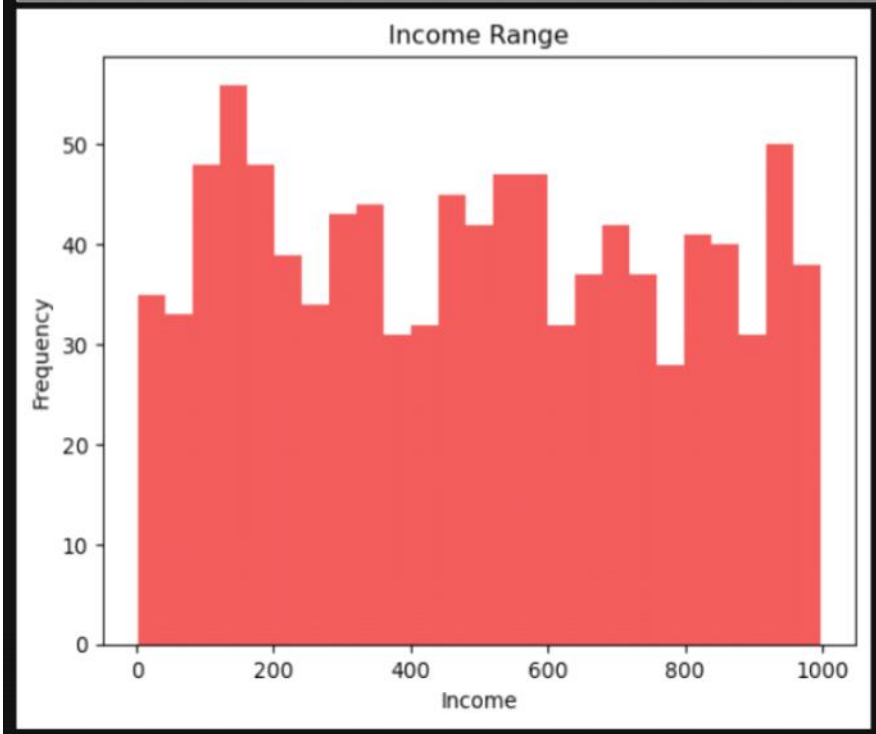
```python
df_ratio['Income'].value_counts()
```

```
Income
106.0    6
925.0    5
709.0    4
255.0    4
137.0    4
        ..
826.0    1
603.0    1
576.0    1
938.0    1
574.0    1
Name: count, Length: 619, dtype: int64
```

```python
df_ratio.describe()
```

	Income
count	1000.000000
mean	491.634000
std	288.712048
min	1.000000
25%	233.250000
50%	488.500000
75%	738.000000
max	999.000000

```python
# Visualization -> Histogram
plt.hist(df_ratio['Income'] , bins = 25, color = '#ED5A5A')
plt.title("Income Range")
plt.xlabel("Income")
plt.ylabel("Frequency")
plt.show()
```




```python
# Discrete -> [Total Number of Childrens]
from scipy import stats
discrete_data = np.random.randint(1,6,1000)
discrete_data
```

```
array([5, 2, 4, 5, 5, 1, 1, 3, 5, 4, 4, 5, 2, 2, 5, 4, 4, 2, 4, 3, 2, 1,
       4, 5, 4, 2, 3, 1, 5, 2, 2, 2, 3, 2, 1, 3, 1, 1, 5, 2, 3, 3, 4, 5,
       1, 5, 5, 4, 1, 4, 4, 3, 2, 2, 3, 4, 1, 4, 3, 2, 2, 5, 3, 4, 5, 5,
       2, 5, 4, 1, 3, 5, 4, 3, 5, 1, 2, 3, 4, 3, 1, 5, 1, 5, 2, 3, 1, 5,
       1, 5, 4, 4, 3, 1, 5, 3, 1, 3, 4, 4, 3, 3, 3, 2, 3, 4, 4, 1, 4, 4,
       5, 5, 5, 4, 4, 5, 2, 3, 1, 3, 4, 1, 4, 3, 4, 3, 3, 1, 5, 4, 4, 5,
       4, 1, 2, 4, 3, 4, 5, 4, 4, 2, 3, 5, 4, 2, 5, 4, 5, 2, 1, 3, 1, 2,
       2, 3, 2, 5, 1, 3, 3, 5, 4, 4, 2, 5, 3, 4, 5, 1, 1, 4, 2, 5, 5, 4,
       4, 4, 5, 3, 2, 2, 2, 2, 1, 2, 5, 1, 5, 1, 5, 5, 4, 3, 5, 5, 2, 2,
       1, 4, 5, 1, 1, 1, 2, 5, 2, 1, 2, 1, 5, 3, 5, 5, 4, 1, 5, 1, 1, 1,
       2, 1, 4, 4, 1, 4, 3, 5, 4, 5, 3, 1, 5, 5, 1, 3, 5, 5, 3, 5, 1, 1,
       5, 2, 4, 3, 5, 1, 1, 3, 2, 5, 3, 1, 4, 3, 2, 2, 5, 3, 4, 1, 1, 2,
       5, 5, 2, 3, 1, 3, 2, 3, 5, 4, 2, 5, 4, 1, 3, 2, 5, 4, 2, 3, 5, 4,
       3, 1, 1, 1, 1, 2, 4, 1, 1, 1, 3, 5, 2, 4, 3, 2, 1, 1, 3, 4, 1, 2,
       5, 1, 5, 1, 1, 4, 2, 4, 1, 4, 4, 2, 3, 4, 3, 5, 5, 4, 5, 4, 1, 3,
       2, 4, 4, 5, 5, 2, 1, 5, 2, 2, 2, 1, 5, 3, 5, 1, 1, 5, 3, 4, 2, 1,
       3, 2, 5, 3, 3, 5, 5, 1, 3, 3, 4, 3, 4, 4, 1, 1, 1, 1, 4, 5, 5, 3,
```

```
df_discrete = pd.DataFrame(discrete_data , columns = ['Total Children'])
df_discrete
```

	Total Children
0	5
1	2
2	4
3	5
4	5
...	...
995	4
996	4
997	3
998	1
999	4

1000 rows × 1 columns

```
df_discrete['Total Children'].value_counts()
```

```
Total Children
5    222
4    214
1    201
2    183
3    180
Name: count, dtype: int64
```

```
mode_discrete = stats.mode(df_discrete['Total Children'])
mode_discrete # 5
```

```
ModeResult(mode=5, count=222)
```

```
mean_discrete = np.mean(df_discrete['Total Children'])
mean_discrete
```

```
3.073
```

```
median_discrete = np.median(df_discrete['Total Children'])
median_discrete
```

```
3.0
```

```
std_discrete = np.std(df_discrete['Total Children'])
std_discrete
```

```
1.4434926393993104
```

```
# Continuous [Height / Weight]
countinous_data = np.random.rand(1000) * 100
continous_data = countinous_data.round(2)
continous_data
```

```
array([2.699e+01, 1.585e+01, 6.186e+01, 4.835e+01, 7.470e+00, 7.281e+01,
       6.821e+01, 8.699e+01, 7.615e+01, 2.028e+01, 7.782e+01, 4.023e+01,
       2.986e+01, 2.214e+01, 8.067e+01, 1.297e+01, 4.650e+01, 4.475e+01,
       8.640e+01, 6.222e+01, 8.311e+01, 1.834e+01, 4.377e+01, 9.000e+01,
       3.511e+01, 2.452e+01, 3.427e+01, 9.521e+01, 1.130e+00, 2.345e+01,
       4.040e+01, 5.972e+01, 6.247e+01, 4.693e+01, 6.163e+01, 2.937e+01,
       3.200e+01, 4.943e+01, 5.613e+01, 9.908e+01, 7.883e+01, 6.496e+01,
       3.619e+01, 8.209e+01, 1.819e+01, 9.973e+01, 8.226e+01, 2.596e+01,
       4.870e+00, 1.112e+01, 8.890e+00, 6.292e+01, 8.628e+01, 2.657e+01,
       6.087e+01, 6.040e+01, 6.035e+01, 7.568e+01, 6.949e+01, 4.951e+01,
       7.731e+01, 9.000e-02, 6.951e+01, 3.893e+01, 3.050e+00, 4.791e+01,
       2.226e+01, 1.155e+01, 7.147e+01, 7.782e+01, 4.587e+01, 2.142e+01,
       2.218e+01, 8.007e+01, 8.747e+01, 8.927e+01, 5.848e+01, 9.788e+01,
       6.707e+01, 9.930e+01, 6.869e+01, 6.954e+01, 5.470e+01, 2.293e+01,
       4.513e+01, 3.003e+01, 7.650e+01, 5.990e+01, 7.708e+01, 4.680e+01,
       8.856e+01, 2.216e+01, 2.800e-01, 1.654e+01, 2.982e+01, 3.489e+01,
       6.200e+00, 7.899e+01, 8.707e+01, 8.483e+01, 7.656e+01, 5.440e+01,
```

```
df_continuous = pd.DataFrame(continous_data, columns = ['Weight'])
df_continuous
```

	Weight
0	26.99
1	15.85
2	61.86
3	48.35
4	7.47
...	...
995	36.83
996	85.30
997	88.95
998	44.17
999	5.98

1000 rows × 1 columns

```
df_continuous['Weight'].value_counts()
```

```
Weight
50.63    3
40.38    3
58.65    3
42.15    2
6.80     2
        ..
42.01    1
24.91    1
11.45    1
52.73    1
5.98     1
Name: count, Length: 948, dtype: int64
```

```
mode_continuous = stats.mode(df_continuous['Weight'])
mode_continuous
```

```
ModeResult(mode=40.38, count=3)
```

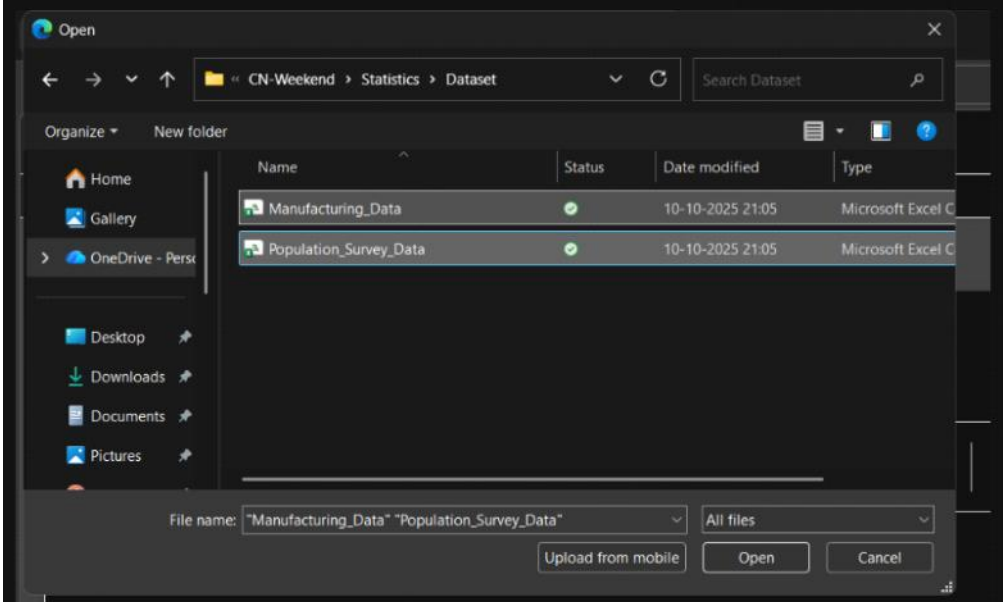
```
mean_continuous = np.mean(df_continuous['Weight'])
mean_continuous
```

```
51.624849999999995
```

```
median_continuous = np.median(df_continuous['Weight'])
median_continuous
```

```
51.305
```

Sampling Techniques



/ anaconda_projects / CN-Python-Weekend / Statistics /

Name

- HeyStats.ipynb
- Manufacturing_Data.csv
- Population_Survey_Data.csv

read_csv

jupyter Population_Survey_Data.csv Last Checkpoint: 5 minutes ago

File Edit View Settings Help

Delimiter: ,

	ID	Age	Gender	Income	Education	Region
1	1	62	Other	96138	Masters	East
2	2	65	Other	76543	PhD	East
3	3	71	Male	138887	Bachelors	East
4	4	18	Other	121643	Bachelors	South
5	5	21	Male	143192	Masters	East
6	6	77	Male	142869	High School	West
7	7	21	Female	125919	Masters	South
8	8	57	Female	140974	High School	North
9	9	27	Other	95040	PhD	East
10	10	37	Female	59105	Masters	North
11	11	39	Male	25241	Masters	West

jupyter Manufacturing_Data.csv Last Checkpoint: 5 minutes ago

File Edit View Settings Help

Delimiter: ,

	ProductID	BatchNumber	ProductionDate	Defective	Weight	Length
1	1	25	2024-01-09	0	462.45374363887805	17.613212046913663
2	2	45	2024-04-18	0	521.7667041961902	15.899233582990265
3	3	20	2024-04-11	0	498.36902973318945	14.177899108335216
4	4	31	2024-06-27	0	488.93876177863234	16.4985991100124
5	5	14	2023-10-27	0	524.4592952384168	15.422275538238395
6	6	8	2023-09-27	0	493.37382727940997	14.478697718606698
7	7	20	2023-12-19	0	511.99973486324683	13.96960325809237
8	8	36	2023-12-02	0	484.63233361404644	17.158433552784466
9	9	30	2024-05-15	0	483.27739746344025	17.954925573306138
10	10	21	2024-03-21	0	536.2116352116723	14.449782552419551

```python
# Sampling Technqiues
import numpy as np
import pandas as pd

df_population = pd.read_csv("Population_Survey_Data.csv")
df_population
```

	ID	Age	Gender	Income	Education	Region
0	1	62	Other	96138	Masters	East
1	2	65	Other	76543	PhD	East
2	3	71	Male	138887	Bachelors	East
3	4	18	Other	121643	Bachelors	South
4	5	21	Male	143192	Masters	East
...	...	...	...	...	...	...
995	996	75	Female	123417	PhD	North
996	997	24	Female	79185	Bachelors	South
997	998	37	Female	57913	PhD	East
998	999	52	Male	69420	Masters	East
999	1000	68	Female	134228	High School	North

1000 rows × 6 columns

```python
# Return a random sample of items from an axis of object.
# Use `random_state` for reproducibility. [np.random.seed(42)]

random_sample = df_population.sample(n=100, random_state = 42)
random_sample
```

	ID	Age	Gender	Income	Education	Region
521	522	74	Male	43185	Bachelors	East
737	738	25	Female	73725	High School	West
740	741	63	Other	110045	Bachelors	South
660	661	25	Male	107801	Bachelors	North
411	412	40	Male	63466	PhD	South
...	...	...	...	...	...	...
436	437	70	Female	71851	PhD	South
764	765	78	Male	116031	Masters	South
88	89	29	Female	21966	PhD	South
63	64	52	Male	104588	Bachelors	North
826	827	55	Male	71836	Bachelors	West

100 rows × 6 columns

```python
df_population['Region'].unique()
```

```
array(['East', 'South', 'West', 'North'], dtype=object)
```

```python
df_population['Region'].value_counts()
```

```
Region
East     257
North    253
West     249
South    241
Name: count, dtype: int64
```

```python
# Stratified Sampling -> Strata[Groups based on Demographics] -> random_sample
stratified_sample = (
    df_population.
    sample(frac = 1 , random_state = 11)
    .groupby('Region')
    .head(25)
)
stratified_sample
```

	ID	Age	Gender	Income	Education	Region
25	26	35	Other	37332	Masters	East
464	465	71	Male	146737	PhD	West
372	373	23	Female	27299	Bachelors	North
730	731	65	Female	95802	Masters	East

```python
stratified_sample['Region'].value_counts()
```

```
Region
East     25
West     25
North    25
South    25
Name: count, dtype: int64
```

```python
stratified_sample[stratified_sample['Region'] == "East"]
```

	ID	Age	Gender	Income	Education	Region
25	26	35	Other	37332	Masters	East
730	731	65	Female	95802	Masters	East
757	758	55	Male	21918	Bachelors	East
357	358	21	Other	61287	Bachelors	East
565	566	72	Female	91965	PhD	East
607	608	54	Male	30463	High School	East
400	401	43	Male	45606	Bachelors	East
617	618	23	Male	22096	PhD	East
309	310	57	Other	123293	High School	East

```python
df_population['Education'].unique()
```

```
array(['Masters', 'PhD', 'Bachelors', 'High School'], dtype=object)
```

```python
df_population['Education'].value_counts()
```

```
Education
PhD            264
Masters        261
Bachelors      245
High School    230
Name: count, dtype: int64
```

```python
# Stratified Sampling -> Strata[Groups based on Demographics] -> random_sample
stratified_sample = (
    df_population.
    sample(frac = 0.5 , random_state = 11)
    .groupby('Education')
    .head(25)
)
stratified_sample
```

	ID	Age	Gender	Income	Education	Region
25	26	35	Other	37332	Masters	East
464	465	71	Male	146737	PhD	West
372	373	23	Female	27299	Bachelors	North
730	731	65	Female	95802	Masters	East

```python
stratified_sample['Education'].value_counts()
```

```
Education
Masters        25
PhD            25
Bachelors      25
High School    25
Name: count, dtype: int64
```

```python
# Clustered Sampling
df_manufacturing = pd.read_csv("Manufacturing_Data.csv")
df_manufacturing
```

	ProductID	BatchNumber	ProductionDate	Defective	Weight	Length
0	1	25	2024-01-09	0	462.453744	17.613212
1	2	45	2024-04-18	0	521.766704	15.899234
2	3	20	2024-04-11	0	498.369030	14.177899
3	4	31	2024-06-27	0	488.938762	16.498599
4	5	14	2023-10-27	0	524.459295	15.422276
...	...	...	...	...	...	...
995	996	44	2023-08-14	0	471.355934	11.495162
996	997	13	2023-11-28	0	526.808606	15.562046
997	998	48	2024-04-15	0	506.841497	16.249218

```
df_manufacturing['Defective'].unique()
```

```
array([0, 1], dtype=int64)
```

```
defective_prod = df_manufacturing[df_manufacturing['Defective'] == 1]
defective_prod
```

	ProductID	BatchNumber	ProductionDate	Defective	Weight	Length
15	16	12	2024-06-01	1	504.755593	14.164331
19	20	48	2024-02-04	1	498.171245	15.282073
53	54	41	2024-02-05	1	495.572190	14.999863
80	81	29	2023-11-16	1	506.766311	17.137462
132	133	29	2023-12-04	1	504.226461	13.115507
154	155	5	2023-12-08	1	519.930161	14.851634
162	163	24	2023-08-16	1	465.627002	10.672669
221	222	24	2024-03-01	1	491.304187	14.652895
249	250	17	2024-03-31	1	515.160761	12.212255

```
defective_prod.shape
```

```
(42, 6)
```

```
defective_prod.shape[0] # 42 defective Products
```

```
42
```

```
clusters= df_manufacturing['BatchNumber'].unique()
clusters
```

```
array([25, 45, 20, 31, 14,  8, 36, 30, 21,  9, 11, 42, 35, 47, 12, 22,  5,
       48, 23,  7, 44, 33, 38,  4, 18,  6, 28,  2, 34, 41, 40, 19, 16, 46,
       49, 43, 17,  3, 13, 10, 29, 32, 24, 26, 39, 15,  1, 27, 37],
      dtype=int64)
```

```
selected_clusters = np.random.choice(clusters , size = 5 , replace=False)
selected_clusters
```

```
array([26, 31, 11, 15, 25], dtype=int64)
```

```
cluster_sample = df_manufacturing[df_manufacturing['BatchNumber'].isin(selected_clusters)]
cluster_sample
```

	ProductID	BatchNumber	ProductionDate	Defective	Weight	Length
0	1	25	2024-01-09	0	462.453744	17.613212
3	4	31	2024-06-27	0	488.938762	16.498599
11	12	11	2024-04-27	0	488.304581	16.396339
49	50	31	2024-04-18	0	524.625071	8.985126
64	65	25	2024-01-23	0	502.540479	14.727805
...	...	...	...	...	...	...
950	951	11	2024-04-26	1	507.839382	14.437701
953	954	25	2024-05-28	0	503.123501	13.413775

```python
# Systematic Sampling => K = N/n -> Kth Factor -> Slicing [::k]
# Pop. Size = 1000 (N) , Sample Size -> 100(n) -> k = 1000/100 -> 10
systematic_sampling = df_population.iloc[::10 , :] # iloc[rows , cols]
systematic_sampling
```

	ID	Age	Gender	Income	Education	Region
0	1	62	Other	96138	Masters	East
10	11	39	Male	25241	Masters	West
20	21	56	Other	121272	PhD	South
30	31	27	Female	58115	High School	East
40	41	53	Female	147482	Bachelors	North
...	...	...	...	...	...	...
950	951	78	Male	121159	PhD	North
960	961	55	Female	34823	Bachelors	South
970	971	36	Other	108570	Bachelors	South
980	981	77	Female	123554	Bachelors	North
990	991	28	Female	139725	Masters	North

100 rows × 6 columns

```python
# Statistical Analysis for Population
mean_income = df_population['Income'].mean()
median_income = df_population['Income'].median()
mode_income = df_population['Income'].mode()
print(mean_income)
print(median_income)
print(mode_income[0])
```

```
85147.959
85589.5
52869
```

```python
df_population['Income'].value_counts()
```

```
Income
59332     2
78883     2
103180    2
52869     2
129352    1
         ..
109901    1
119304    1
146887    1
41272     1
134228    1
Name: count, Length: 996, dtype: int64
```

```python
df_population.describe()
```

	ID	Age	Income
count	1000.000000	1000.000000	1000.000000
mean	500.500000	48.167000	85147.959000
std	288.819436	18.009427	38089.505827
min	1.000000	18.000000	20107.000000
25%	250.750000	33.000000	51922.500000
50%	500.500000	48.000000	85589.500000
75%	750.250000	64.000000	118648.500000
max	1000.000000	79.000000	149870.000000

```python
# Evaluating the Population Mean Vs Sample Mean from various Sampling Techniques

actual_mean_income = df_population['Income'].mean()
print(f"Actual Mean: {actual_mean_income}")

# Random Sample Technique
random_sample_mean_income = random_sample['Income'].mean()
print(f"Random Sample Mean: {random_sample_mean_income}")

# Stratified Sample Technique
stratified_sample_mean_income = stratified_sample['Income'].mean()
print(f"Stratified Sample Mean: {stratified_sample_mean_income}")

# Systematic Sample Technique
systematic_sample_mean_income = systematic_sampling['Income'].mean()
print(f"Systematic Sample Mean: {systematic_sample_mean_income}")

# Cluster Sample Technique
cluster_sample_mean_income = cluster_sample['Income'].mean()
print(f"Cluster Sample Mean: {cluster_sample_mean_income}")
```

```
Actual Mean: 85147.959
Random Sample Mean: 83263.23
Stratified Sample Mean: 90655.28
Systematic Sample Mean: 88961.4
Cluster Sample Mean: 81986.44144144144
```